

Agile Practices for the Global Teaming Model

John Noll, Abdur Razzak, Ita Richardson, Sarah Beecham

Lero, the Irish Software Research Centre

University of Limerick, Ireland

{john.noll, abdur.razzak, ita.richardson, sarah.beecham}@lero.ie

Abstract—

The Global Teaming Model (GTM) is a CMMI®-style process model comprising 64 recommendations that synthesizes empirical evidence on how to overcome the obstacles faced by global software development projects.

While the Global Teaming Model recommendations specify *what* a global software development project should do, it does not specify *how*. In order to provide concrete guidance for projects that wish to employ Agile methods in a global software development context, we asked, could the practices described in the Scaled Agile Framework® (SAFe®) provide examples for how GTM recommendations could be realized?

We found 79 of 90 SAFe® Team level practices partly or wholly implement GTM recommendations. The SAFe® mapping onto the Global Teaming model provides a concrete roadmap for teams who need specific advice on how to implement GTM practices that have been identified as necessary to their success.

Index Terms—Global Software Development; Empirical Software Engineering; Scaling agile; Scaled Agile Framework®; SAFe®.

I. INTRODUCTION

The Global Teaming Model (GTM) [28] is a CMMI®-style [31] process model comprising 64 recommendations that synthesizes empirical evidence on how to overcome the obstacles faced by global software development projects (Fig. 1). While the Global Teaming Model recommendations specify *what* a global software development project should do, it does not specify *how*. This is intentional, as factors such as development method and target domain mean different implementations are required for different contexts.

In order to provide concrete guidance for projects that wish to employ Agile methods in a global software development context, we asked, could the practices described in the highly detailed Scaled Agile Framework® (SAFe®) [19] provide examples for how GTM recommendations could be realized?

To answer this question, we first identified 90 SAFe® practices that are specified in the “Team” level of SAFe®. Then, we mapped these practices onto GTM recommendations they might implement. We found 79 of 90 SAFe® Team level practices partly or wholly implement GTM recommendations.

The SAFe® mapping onto the Global Teaming Model provides a concrete roadmap for teams who need specific advice on how to implement GTM practices that have been identified as necessary to their success.

II. BACKGROUND

A. Global Software Development and Agile methods

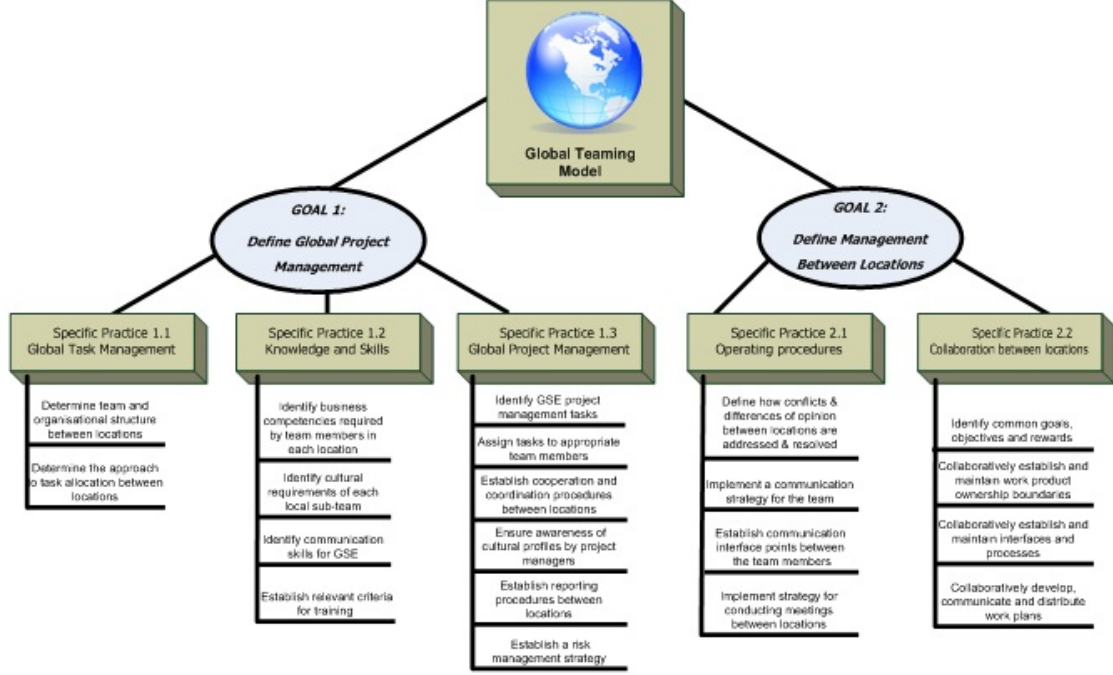
Improved communication technologies, access to global talent, cheaper labor, proximity to new markets and legal requirements have all contributed to the growth in Global Software Development (GSD) [32]. GSD is software work undertaken in different geographical locations, across national boundaries in a coordinated fashion through synchronous and asynchronous interaction [15]. GSD is reputed to suffer from communication breakdowns, low morale and delays due to teams being geographically, culturally and temporally separated [6, 7, 22]. Over the years several recommendations have been published in support of this complex development paradigm [8]. Specifically, complete solutions have been offered in the form of process models and frameworks [10, 18, 28].

Traditionally GSD has followed a plan driven, structured, waterfall approach, where tasks are allocated according to where they appear in the software development lifecycle [11, 21]. It was considered that agile methods envisaged for *small projects* and *co-located teams* [2, 16] would be a poor fit for GSD because both Agile and distributed development approaches differ significantly [27]. Agile methods tend to rely on informal processes and regular face-to-face communication to facilitate coordination, whereas distributed software development relies on formal mechanisms. Yet there is a growing trend for companies to adopt agile methods as reported in a tertiary study of GSD [14]. Adopting Agile practices such as short iterations, frequent builds, and continuous delivery all pose challenges to configuration management and version management [25]. But, practices such as *Short iterations* increase transparency of Work-in-Progress (WIP) and provide a big picture project progress to stakeholders [24]. However, introducing an Agile method can change the culture (command and control model) in a company; so to implement the Agile practices in global software environment developers need to have more autonomy as well as decision-making power [12].

B. Agile Frameworks

Scaling agile covers the movement from a few agile teams to multiple agile development teams, where the number of teams can be in the hundreds [3]. Scott Ambler [3] pointed out several factors that need to be considered when scaling agile such as team size, geographical distribution, entrenched culture, system complexity, legacy systems, regulatory compliance, organizational distribution, governance and enterprise

Fig. 1. The Global Teaming Model.



focus. In general, productivity and quality are the two main concerns of any organization when adopting a scaling agile paradigm.

The choice of scaling agile framework adopted or how the framework is tailored will depend on the organization's size or on "what works" based on their own business goals, operative model, and needs. The Agile Scaling Knowledgebase (ASK) (<http://www.agilescaling.org/home.html>) developed a matrix of different Agile frameworks namely *Scrum-of-Scrum (SoS)*, *Large Scale Scrum (LeSS)*, *Scaled Agile Framework (SAFe)*, *Disciplined Agile Delivery (DAD)*, *Spotify Model*, and *Scrum at Scale*. This matrix shows that SAFe®, launched in 2012 by Dean Leffingwell [19], focuses on large enterprises and takes a scaled approach to Agile adoption. For this reason we have chosen SAFe® as the source of concrete guidance on how to extend the Global Teaming Model for Agile development.

C. Scaled Agile Framework

The Scaled Agile Framework® (SAFe®) is essentially a container for several existing Agile approaches, and is primarily developed for organizing and managing agile practices in large enterprises. Early adopters of SAFe® report that the application of the practices contained in this framework led to significant productivity and quality improvements [17]. SAFe® adoption is widespread to include sectors such as manufacturing, software, financial services. Yet among these cases, we have only found one organization that adopted SAFe® in a distributed environment [1].

The SAFe® framework includes [19, 20]: a process model that covers the highest and the lowest level in the enterprise; Associated Agile values and practices, including Scrum [29], eXtreme Programming [5], Kanban and Lean Software Development [26], and the Agile Manifesto [13]; four core values: code quality, alignment, program execution and transparency.

SAFe® 3.0 (<http://scaledagileframework.com>) is organized into three layers:

- 1) *Portfolio* – Funding and coordinating programs.
- 2) *Program* – Contains 5-12 teams working towards a common goal.
- 3) *Team* – Teams, which practice both Scrum and eXtreme Programming, or Kanban (or all three).

Teams are collectively responsible for defining, building and testing software in fixed-length iteration and releases. The team events (Backlog Refinement, Sprint Planning, Sprint Review) are an integral part of SAFe® and help to reduce coordination overheads between teams. These teams typically consist of 7-9 members and teams operate on an identical cadence and iteration lengths in order to provide better integration among teams [30]. But, adoption of only Scrum at the team level could lead to additional problems in task synchronization. To resolve this issue, SAFe® introduces the *Release Planning* meeting to synchronize team tasks after every five iterations.

D. Global Teaming Model

The Global Teaming Model (GTM) is a model for global software engineering, with a particular emphasis on organi-

zation, governance and management of globally distributed development teams [28]. The Global Teaming Model follows the hierarchical structure and nomenclature of the CMMI® [31]. At the highest level there are two broad goals, “Define Global Project Management” and “Define Management Between Locations.” These goals are decomposed into Specific Practices that define broad categories of practice that lead to the parent goals.

Specific Practices are further elaborated into Sub-practices. Finally, Sub-practices have one or more recommendations that specify detailed actions to be taken. In total, the GTM has five Specific Practices, twenty Sub-practices, and 64 recommendations, that have been validated against a real industrial case [6, 9].

The first GTM goal –“Define Global Project Management” –recognizes that global project management, while encompassing the expected tasks of any project management setting, must also include new tasks related to managing a virtual software engineering team comprising distributed individuals, teams, suppliers, communities, etc. The first goal comprises three Specific Practices: “Global Task Management,” which defines how distributed teams are organized into a virtual organization structure, and how tasks are allocated to teams; “Knowledge and Skills,” which concerns both project knowledge management and staff training; and “Global Project Management,” which defines decision-making roles and responsibilities in the virtual organization.

The second goal, “Define Management Between Locations,” focuses on communication and collaboration among distributed participants. This is achieved through two Specific Practices: “Operating Procedures,” which is concerned with the way communication between teams, and meetings involving more than one team, are conducted; and “Collaboration Between Locations,” which focuses on how distributed teams work together to set objectives, establish responsibilities, and plan work.

E. A Global Teaming Model for Agile Development

The Scaled Agile Framework® (SAFe®) offers practitioners a dynamic and detailed view of Agile and Lean principles but does not cover all aspects of agility required in a distributed environment context. On the other hand, while the GTM places particular emphasis on the organization and management of globally distributed development teams, it does not specify *how* to develop software using Agile and Lean principles. Furthermore, the GTM recommended practices are normative, and do not prescribe how to implement the practice. So, we hypothesize that combining SAFe practices at the Team level, together with GTM recommendations will provide practitioners with a framework of implementable practices. Thus, our research going forward is driven by following hypothesis:

The Scaled Agile Framework® can provide concrete implementations of GTM recommendations for Agile development.

III. METHOD

The Scaled Agile Framework® is a comprehensive framework that integrates hundreds of practices from several Agile and Lean software development methods. As such, we decided to focus on a subset of SAFe® – the *Team* level – to test our hypothesis that SAFe® can provide implementations of Global Teaming Model practices.

SAFe® is described in some detail on the SAFe® website (www.scaledagileframework.com). To identify SAFe® Team level practices, we started at the SAFe® home page, which contains a “rich pictures [4]” diagram that serves as an image map index of SAFe®. Each link in the map leads to a page describing the element (procedure, role, or set of practices) depicted by that section of the map.

We followed each index link from the Team level of the index, and extracted practices listed on that page and related pages. We then organized these practices into a hierarchy mirroring the organization of web pages.

Once we had a comprehensive catalog of SAFe® Team level practices, we then compared each SAFe® practice to the set of GTM recommendations. If a SAFe® practice could be considered to contribute to the implementation of a GTM practice, the SAFe® practice was added as a “Sub-recommendation” of the GTM recommendation.

The end result was a version of the Global Teaming Model with some recommendations elaborated a further level to include SAFe® implementation practices.

IV. RESULTS: SAFe® PRACTICES FOR GTM

We identified a total of 90 practices at the SAFe® Team level. Of these, 79 represent ways to implement Global Teaming Model recommendations.

Table I shows the specific GTM recommendations that have associated implementation practices from SAFe®. GTM recommendations are identified by a numeric identifier reflecting their position in the GTM’s hierarchical organization. Relevant SAFe® practices are listed below the GTM practice they implement. GTM recommendations that do not have associated SAFe® practices are omitted to save space.

As can be seen in Table I, in total, 15 recommendations are implemented by SAFe® Team level practices. The majority of these (11) address GTM recommendations under Specific Goal 2 - “Define Management Between Locations.” This is not surprising, as the SAFe® Team level concerns itself with what individual teams do, and the GTM’s Specific Goal 2 is concerned with how individual teams work with other teams, as opposed to the global project view.

In many cases, SAFe® practices only facilitate the implementation of a GTM recommendation. For example, GTM Recommendation 1.1.2.3 “Retain tasks that require frequent communication between groups within collocated teams” has two associated SAFe® practices: “Break larger initiatives into stories to ease estimation of larger work items” and “Break each story into tasks then estimate and identify dependencies of a specific task.” However, these practices are only concerned with the granularity of tasks, and not with how they are

allocated. In the first practice, smaller tasks are likely to be easily implemented by a single team, so breaking larger initiatives into stories will lead to smaller tasks. Identifying dependencies allows tasks that depend on other tasks to be allocated to the same team as the task depended on. In either case, a further step is required to determine whether frequent communication would be required among teams implementing dependent tasks; if not, there is less need for those tasks to be allocated to the same team.

In some cases, the aggregate of SAFe® practices fully implements a GTM recommendation. For example, GTM Recommendation 2.2.2.3 “Each location should understand how their modifications to the product unit can affect the other locations” has six associated SAFe® Team practices, related to continuous integration and synchronized sprints. While these practices are not the only way to convey how changes affect other teams, they will certainly do so in a dramatic way on the first occasion that a team makes a modification that breaks the build.

V. CONCLUSIONS

In this paper, we have demonstrated how Scaled Agile Framework® practices can provide implementation guidance for Global Teaming Model recommendations, by showing where SAFe® practices fit into the GTM’s hierarchical organization. This guidance is appropriate for projects that want to implement Agile methods in a global software development context.

In future work, we intend to validate the SAFe® practice mapping to the GTM by employing this mapping in an actual industrial development project [23].

Acknowledgments This work was supported, in part, by Enterprise Ireland and the European Development Fund through grant IR20130022, and by Science Foundation Ireland grants 10/CE/I1855 and 13/RC/2094 to Lero - the Irish Software Research Centre.

REFERENCES

- [1] Applying the Scaled Agile Framework® in an outsourcing context: Infogain case study. <http://scaledagileframework.com/infogain-case-study/>, accessed 2016-04-13, 2014.
- [2] Pekka Abrahamsson, Kieran Conboy, and Xiaofeng Wang. “Lots done, more to do”: the current state of agile systems development research. 2009.
- [3] Scott W. Ambler. Agile software development at scale. In *Balancing agility and formalism in software engineering*, pages 1–12. Springer, 2008.
- [4] D.E. Avison and H.U. Shah. Towards an SSM toolkit: rich picture diagramming. *European Journal of Information Systems*, 1(6):397–408, 1992.
- [5] Kent Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional, 2000.
- [6] Sarah Beecham. Motivating software engineers working in virtual teams across the globe. In *Software Project Management in a Changing World*, pages 247–273. Springer, 2014.
- [7] Sarah Beecham and John Noll. What motivates software engineers working in global software development? In *Product-Focused Software Process Improvement*, pages 193–209. Springer, 2015.
- [8] Sarah Beecham, Pdraig O’Leary, Ita Richardson, Sean Baker, and John Noll. Who are we doing global software engineering research for? In *8th International Conference on Global Software Engineering (ICGSE)*, pages 41–50. IEEE, 2013.
- [9] Sarah Beecham, Ita Richardson, and John Noll. Assessing the strength of global teaming practices: A pilot study. In *10th International Conference on Global Software Engineering (ICGSE)*, pages 110–114. IEEE, 2015.
- [10] Sadhana Deshpande, Sarah Beecham, and Ita Richardson. Using the PM-BOK guide to frame GSD coordination strategies. In *8th International Conference on Global Software Engineering (ICGSE)*, pages 188–196. IEEE, 2013.
- [11] Hans-Christian Estler, Martin Nordio, Carlo A Furia, Bertrand Meyer, and Johannes Schneider. Agile vs. structured distributed software development: A case study. *Empirical Software Engineering*, 19(5):1197–1224, 2014.
- [12] Martin Fowler. Using an agile software process with offshore development. <http://www.martinfowler.com/articles/agileOffshore.html>, accessed 2016-04-15, 2006.
- [13] Martin Fowler and Jim Highsmith. The agile manifesto. *Software Development*, 9(8):28–35, 2001.
- [14] Geir K. Hanssen, Darja Smite, and Nils Brede Moe. Signs of agile trends in global software engineering research: A tertiary study. In *Sixth International Conference on Global Software Engineering Workshops (ICGSEW)*, pages 17–23. IEEE, 2011.
- [15] James D. Herbsleb and Deependra Moitra. Global software development. *IEEE Software*, 18(2):16–20, 2001.
- [16] Tuomo Kähkönen. Agile methods for large organizations – building communities of practice. In *Agile Development Conference*, pages 2–10. IEEE, 2004.
- [17] Maarit Laanti. Characteristics and principles of scaled agile. In *Agile Methods. Large-Scale Development, Refactoring, Testing, and Estimation*, pages 9–20. Springer, 2014.
- [18] Ansgar Lamersdorf, Jürgen Münch, and Dieter Rombach. A survey on the state of the practice in distributed software development: Criteria for task allocation. In *Fourth International Conference on Global Software Engineering (ICGSE 2009)*, pages 41–50. IEEE, 2009.
- [19] Dean Leffingwell. Scaled Agile Framework® 3.0. <http://v3.scaledagileframework.com/>, accessed 2016-04-15, 2015.
- [20] Richard Levy, Michael Short, and Peter Measey. Agile foundations: Principles, practices and frameworks. BCS, 2015.
- [21] Bertrand Meyer, Jerzy R. Nawrocki, and Bartosz Walter. *Balancing Agility and Formalism in Software Engineering*, volume 5082. Springer Science & Business Media, 2008.
- [22] John Noll, Sarah Beecham, and Ita Richardson. Global software development and collaboration: barriers and solutions. *ACM Inroads*, 1(3):66–78, 2010.
- [23] John Noll, Sarah Beecham, and Ita Richardson. Global teaming for global software development governance: A case study. In *International Conference on Global Software Engineering*, Irvine, California, USA, August 2016 (forthcoming).
- [24] Maria Paasivaara and Casper Lassenius. Using iterative and incremental processes in global software development. In *3rd International Workshop on Global Software Development*, pages 42–47, 2004.
- [25] Maria Paasivaara and Casper Lassenius. Could global software development benefit from agile methods? In *International Conference on Global Software Engineering (ICGSE’06)*, pages 109–113. IEEE, 2006.
- [26] Mary Poppendieck and Tom Poppendieck. *Lean Software Development: An Agile Toolkit: An Agile Toolkit*. Addison-Wesley, 2003.
- [27] Balasubramaniam Ramesh, Lan Cao, Kannan Mohan, and Peng Xu. Can distributed software development be agile? *Communications of the ACM*, 49(10):41–46, 2006.
- [28] Ita Richardson, Valentine Casey, Fergal McCaffery, John Burton, and Sarah Beecham. A process framework for global software engineering teams. *Information and Software Technology*, 54(11):1175–1191, 2012.
- [29] Ken Schwaber and Mike Beedle. *Agile Software Development with Scrum*. Prentice Hall, 2002.
- [30] Igor Stojanov, Oktay Turetken, and Jos J.M. Trienekens. A maturity model for scaling agile development. In *41st Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 446–453. IEEE, 2015.
- [31] CMMI Product Team. Capability Maturity Model® Integration for development. *Software Engineering Institute Technical Report CMU/SEI-2006-TR-008*, 2006.
- [32] Aurora Vizcaino, Felix García, Mario Piattini, and Sarah Beecham. A validated ontology for global software development. *Computer Standards & Interfaces*, 46:66–78, 2016.

APPENDIX

TABLE I: SAFe practice mapping to GTM recommendations.

SG = GTM “Specific Goal”; SP = GTM “Specific Practice”; Sub P. = GTM “Sub Practice”; R = GTM “Recommendation”; • = SAFe® Team-level practice

ID	Description
SG 1	Define Global Project Management
SP 1.1	Global Task Management
Sub P. 1.1.2	Determine the approach to task allocation between locations
R 1.1.2.3	Retain tasks that require frequent communication between groups within collocated teams. • Break larger initiatives into stories to ease estimation of larger work items • Break each story into tasks then estimate and identify dependencies of a specific task.
SP 1.3	Global Project Management
Sub P. 1.3.1	Identify GSE project management tasks
R 1.3.1.1	Define ability and potential productivity of team: Global project manager should allocate tasks and timescales that are realistic. Where possible, the project manager should be actively involved in the recruitment and selection of team members. Failing this, they should gather all information relating to the technical and professional experience of potential and existing team members. When teams are in place and project details reported project managers should understand and document how individuals contribute to that project along with their skills and knowledge. • Measure the team’s capacity (based on previous sprint) to establish the velocity for the upcoming iteration • Estimate initial capacity to establish a common starting point for team estimation • Calculate team “delivered velocity” by multiplying the throughput by an average story size • Use Capacity allocation technique to make a policy decision • Build an approximation of team’s current process and define some initial Work-in-Process Limits (WIP) limits
Sub P. 1.3.4	Establish cooperation and coordination procedures between locations
R 1.3.4.1	Ensure that a suitable infrastructure, process and management procedures are in place to help establish cooperation and coordination between locations. Achievable milestones should be planned and agreed. Projects should be monitored with reference to costs, time, productivity, quality and risk. • Measure the team’s capacity (based on previous sprint) to establish the velocity for the upcoming iteration • Perform Program Increment (PI) planning to understand and agree on one or more iteration goal(s) based on the team and PI Objectives • Build an approximation of teams current process and define some initial WIP limits • Estimate initial capacity to establish a common starting point for team estimation • Use objective measure that includes average lead time (how long it takes, on average to get through the system), WIP (the average number of items in the system), and throughput (the number of stories completed per a period of time) by Cumulative Flow Diagram (CFD) • Differentiate between Features and Objectives to reduce the confusion • Stretch objectives to provide a way to help that ensure the delivery time-box will be met • Write SMART Objectives • Assign business value to each of the teams individual objectives [Communicate Business Value with Objectives] • Finalize the Team PI Objectives with business value and stretch objectives • Commit to PI Objectives by agreeing to do everything in the team’s power to meet the committed objectives • Create Program and Value Stream Objectives • Decrease excess WIP to reduce overhead and thrashing, and it increase productivity and velocity. [Shed Excess WIP with Realistic Objectives] • Planning at regular PI intervals limits variances to a single PI time-box, thereby increasing Agile Release Train and Value Stream predictability [Use a regular cadence to limit the accumulation of variance]
Sub P. 1.3.5	Establish reporting procedures between locations
R 1.3.5.1	Regular formal reporting will help the project manager to remain aware of how project is progressing. Procedure should include and encourage team members to report whether or not they can take on that task in the given time and report any problems before it is too late. • Demonstrate completed story and summarize team’s increment
SG 2	Define Management Between Locations
SP 2.1	Operating Procedures
Sub P. 2.1.2	Implement a communication strategy for the team
R 2.1.2.1	Plan, facilitate, encourage and monitor communication between teams. • Perform constant communication and collaboration (for fast, effective, and empowered decision making, and to help individuals meet their responsibilities). • Perform Daily Stand-up to understand team’s status, escalate problems, and get help from other team members • Use “big visual information radiator” (BVIRs) to understand and track progress during iteration execution • Use story board to visualize the stories and their progress throughout the iteration • PI planning, iteration planning, backlog refinement, inspect and adapt, architecture discussion, etc., all benefits from frequent meetings [Schedule frequent meetings using a predictable cadence]

Continued on next page.

ID	Description
Sub P. 2.1.3	Establish communication interface points between the team members
R 2.1.3.2	Ensure that relevant team members are made aware of how and when they will receive inputs to products, needs to distribute outputs from and when complete work products are required. • Create “pull” process within the iteration and continuously balance the work to increase throughput team by using WIP • Manage dependencies and resolve impediments by continuously and actively engaging with other teams
SP 2.2	Collaboration between locations
Sub P. 2.2.1	Identify common goals, objectives and rewards for the global team
R 2.2.1.3	Project goals and objectives communicated, understood and agreed across all team members regardless of location. • Facilitate trust among team members by having a common mission, common iteration goals, and team Program Increment (PI) objectives • Team plan together to create a common vision and roadmap
Sub P. 2.2.2	Collaboratively establish and maintain work product ownership boundaries
R 2.2.2.2	Each location should understand their role within the life cycle of the product. • Team plan together to create a common vision and roadmap
R 2.2.2.3	Each location should understand how their modifications to the product unit can affect the other locations. • Team integrates and demonstrates together • Teams integrate and evaluate (at least) on iteration boundaries; program and value streams integrate and evaluate on PI boundaries. [Apply nested cadence harmonic multiples to synchronize work] • Implement feature and component Level continuous integration • Enable continuous integration • Common Cadence • Make CI a Culture
Sub P. 2.2.3	Collaboratively establish and maintain interfaces and processes
R 2.2.3.1	Define common process goals across all locations. • Individual Agile Teams are aligned to common iteration lengths [Exploit economic of scale by synchronizing work from multiple projects] • Teams are aligned to common time-boxes and similar batch sizes [To reduce queues, synchronize the batch size and timing of adjacent processes] • Test-first • Use acceptance Test-Driven Development • Refactor • Create a Community of Practices (CoP) for software quality/continuous integration/test first/refactoring • Work in Pairs • Develop a collective ownership • Build in quality in the Firmware and Hardware • Use exploratory early iterations • Frequent integrate and test at System-Level • Apply design verification
R 2.2.3.3	Seek and encourage input from team members at all locations. • Improve continuous collaboration by using regular feedback loops.
R 2.2.3.4	Let team members know their input to process development and ownership is valued. • Perform Iteration Retrospective to identify way to improve • Team learn together and share best practices through inter-team communication supported by Communities of Practice (CoP).
Sub P. 2.2.4	Collaboratively develop, communicate and distribute work plans.
R 2.2.4.1	Achievable milestones should be planned and agreed. • In order to reliably meet PI objectives and planning iteration is unplanned and provides schedule margin [Provide sufficient capacity margin to enable cadence] • Short iterations help to control the number of stories in the iteration batch [Use a regular cadence to enable small batch sizes]
R 2.2.4.3	Contingency plans should be in place to address potential risks. • If a feature doesn't make it into a PI (or release) and it remains high priority, its delivery can be anticipated to be on schedule in the next PI [Use cadence to make waiting time predictable] • Teams plan with stretch objectives; these are sacrificed as necessary when plan meet reality [Capacity margin enables synchronization of deliverables] • Consider challenges and trade-offs
R 2.2.4.4	Establish procedures to coordinate implementation of contingencies when and if required. • Perform backlog refinement to underline the problems of current plan • Synchronize with Supplier and Solution Context